FP game design Waizy Giesberts

Masheed Waizy
Kevin Giesberts, 7254970

October 2026

1 Introduction and Rules

This document describes the implementation of the game, Asteroids. Asteroids is a real-time single-player arcade shooter, played in a two-dimensional wrap-around space environment. The player controls a spaceship that can rotate and thrust forward, firing projectiles to break apart moving asteroids and UFOs. The game ends when the player loses all three lives by colliding with an asteroid or an enemy flying saucer.

2 Design

2.1 Data structures

2.1.1 The Game State

The full state of our game is represented using a Game State data type:

```
data GameState = GameState { ship :: Ship
                           , asteroids :: [Asteroid]
                           , score :: Int
                           , lives :: Int
                           , Alive :: Bool
                           , bullets [Bullet]
                           }
```

The player will always be human.

2.1.2 The player and the field

The player, projectiles, and asteroids exist in a flat space. Positions and velocities will be represented by 2D vectors (tuples of floats):

```
type Vector = (Float, Float)
data Ship = Ship { position :: Vector, velocity :: Vector, heading :: Float }
data AsteroidSize = Large | Medium | Small
```

```
data Asteroid = Asteroid { astPos :: Vector,
                           astVel :: Vector,
                           size :: AsteroidSize }
data Bullet = Bullet
{ bulletPos  :: Vector
, bulletVel  :: Vector
, bulletLifeTime :: Float
} deriving (Show)
```

The board configuration is a continuous coordinate system that wraps around the edges of the screen. If the player goes out of bounds, then he should respawn at the other side of the screen with the same velocity and direction.

2.1.3 Movement

Arrow-keys for moving. Up for thrust. Left arrow to turn left and right arrow to turn right. Even after you apply thrust you keep moving. If the player goes out of bounds, then the spaceship should be spawned on the other side of the screen, but with the same momentum. The spaceship should shoot with space, in the direction that it is facing.

2.2

2.3 Interface

3 Implementation of the Minimum Requirements

Player. The player controls (at least) one character, using the keyboard. Enemies. The game has antagonistic characters in some sense, which try to make you lose the game. You should implement a minimal "intelligence" for those enemies.

3.1

Randomness. The game has to include random components: the place where new enemies appear, the place where bonus are located, and things like that.

3.2 Animation

3.2.1 Menu

- Hovering on a button will show a hover animation.

3.2.2 In Game

- When the player applies thrust a fire animation plays behind the space ship.

- When the player destroys an enemy a destruction animation will play.
- When the player dies an animation plays where he himself gets destroyed, after which a transition animation plays to the menu with the score of the game.

3.3 Pause

The player can pause the game by pressing P. They then see a pause menu from which they can either continue the game, restart the game, exit to main menu, or exit to desktop.

3.4 Interaction with the file system

The game score of every game is stored in a JSON file on the players computer. The key is the Unix timestamp of when the game was started and the value is the score of the game.

```
{
    "1791895993": "440"
}
```

On the highscore page you can see all your scores and when you reached them.

4 Implementation of the optional Requirements

We have no plans to implement any of the optional requirements. This is subject to change in the future.